

Manage Account API Documentation

1. Overview

Base Router Path: [/account](#)

This section covers the account management endpoints, allowing Administrators and Members to view and modify user account information as permitted by the **R3 User Account Rule** in the ManageAcct use case.

1.1 Interface Definitions

- **IUser** (from [common/user.interface.ts](#))
 - **credentials**: { username: string, password: string }
 - **_id?**: string
 - **email**: string
 - **agreed**: boolean
- **IUserAccount** (extends IUser)
 - **status**: IAccountStatus
 - **privilegeLevel**: IPrivilegeLevel
- **IAccountStatus**
 - **'Active'** | **'Inactive'**
- **IPrivilegeLevel**
 - **'Administrator'** | **'Coordinator'** | **'Member'**
- **ISuccess<T>** (from [common/server.responses.ts](#))
 - **name**: SuccessName
 - **message?**: string
 - **authorizedUser?**: string
 - **payload**: T | null

2. REST API Endpoints

Method	Path	Function	Resource or Response Type	Body Type
GET	/users/:username	Get user account details	ISuccess (IUserAccount payload)	<i>None</i>

PATCH	/users/:username/status	Update account status	ISuccess (IUserAccount payload)	{ status: IAccountStatus }
PATCH	/users/:username/privilege	Update privilege level	ISuccess (IUserAccount payload)	{ privilegeLevel: IPrivilegeLevel }
PATCH	/users/:username/username	Update username	ISuccess (IUserAccount payload)	{ newUsername: string }
PATCH	/users/:username/email	Update email	ISuccess (IUserAccount payload)	{ email: string }
PATCH	/users/:username/password	Update password	ISuccess (IUserAccount payload)	{ currentPassword, newPassword }

3. Request Payload Details

3.1 Get User Account (GET /account/users/:username)

- **Path Parameters:** **username** (string, required)
- **Headers:** **Authorization:** Bearer token (JWT)
- **Authorization Rules:** Administrators can view any user; Members can only view their own.

3.2 Update Account Status (PATCH /account/users/:username/status)

- **Body Fields:** **status** (IAccountStatus, required)
- **Authorization:**
 - Administrators: Any account.
 - Members: Own account only.
 - **Rule (R1):** Last Administrator cannot be inactivated.

3.3 Update Privilege Level (PATCH /account/users/:username/privilege)

- **Body Fields:** `privilegeLevel` (IPrivilegeLevel, required)
- **Authorization:** Only Administrators. Members are strictly forbidden.

3.4 Update Username (PATCH /account/users/:username/username)

- **Body Fields:** `newUsername` (string, required)
- **Authorization:** Members only (own account). Administrators cannot change usernames.
- **Validation:** 4+ chars, not banned, unique (case-insensitive).

3.5 Update Email (PATCH /account/users/:username/email)

- **Body Fields:** `email` (string, required)
- **Authorization:** Members only (own account).
- **Validation:** Must meet domain eligibility (e.g., cmu.edu).

3.6 Update Password (PATCH /account/users/:username/password)

- **Body Fields:** `currentPassword` (required for Members), `newPassword` (required).
- **Authorization:** Administrators can change any password (no current pass needed); Members can only change their own.

4. Response Payload Details (HTTP 200)

- **AccountRetrieved:** Returns full `IUserAccount` (password obfuscated).
- **StatusUpdated:** Returns updated user account with new status.
- **PrivilegeUpdated:** Returns updated user account with new privilege level.
- **UsernameUpdated:** Returns updated user account with new username.
- **EmailUpdated:** Returns updated user account with new email.
- **PasswordUpdated:** Returns updated user account (password obfuscated).

5. Active/Inactive Behavior (R5)

1. **Default:** Accounts are 'Active' by default.
2. **Auto-Logout:** If a logged-in user is set to 'Inactive', they are logged out immediately and emailed.
3. **Login Block:** Inactive users cannot log in.
4. **Reactivation:** Only an Administrator can reactivate an account.
5. **Visibility:** Inactive account resources are hidden from all users (except Admins).

6. Privilege Change Behavior (R4)

Changes take effect upon the **next login**.

Privilege Level	Permissions
Member	Standard Member use cases.
Coordinator	Member + Coordinator use cases.
Administrator	Member + Coordinator + Admin use cases.

7. Initial Administrator (R2)

Field	Value
Username	Admin
Password	admin
Privilege Level	Administrator
Status	Active

8. Error Names

Error Name	Description
------------	-------------

UnauthorizedRequest	User lacks permission for the action.
LastAdministrator	Cannot inactivate the sole Administrator.
UserNotFound	Target user does not exist.
InvalidPassword	Does not meet complexity rules.
UsernameExists	New username is already taken.

9. Interface Extensions Required

File: **common/user.interface.ts**

TypeScript

```
export type IAccountStatus = 'Active' | 'Inactive';
export type IPrivilegeLevel = 'Administrator' |
'Coordinator' | 'Member';

export interface IUserAccount extends IUser {
  status: IAccountStatus;
  privilegeLevel: IPrivilegeLevel;
}
```

File: **common/server.responses.ts** (SuccessName additions)

AccountRetrieved, StatusUpdated, PrivilegeUpdated, UsernameUpdated,
EmailUpdated, PasswordUpdated.

File: **common/socket.interface.ts** (Socket.io event type extensions)

TypeScript

```
import { IUserAccount } from './user.interface';
```

```

export interface ServerToClientEvents {
  ping: () => void;
  accountUpdated: (account: IUserAccount) => void;
  forceLogout: (reason: string) => void;
}

export interface ClientToServerEvents {
  ping: () => void;
  subscribeAccount: (username: string) => void;
  unsubscribeAccount: (username: string) => void;
}

```

10. Socket.io Real-Time Events

10.1 Overview

Socket.io is used alongside the REST API to provide real-time, bidirectional updates for account management. This ensures that when multiple clients (e.g., an Administrator and a Member) are viewing or editing the same account, changes made by one party are immediately reflected on the other's screen. It also enables forced logout when an account is inactivated.

Connection: Clients connect to the Socket.io server with a JWT token passed as a query parameter (?token=<JWT>). The server validates the token before accepting the connection (existing behavior in app.ts).

10.2 Server-to-Client Events

Event Name	Payload	Description
accountUpdated	IUserAccount (password obfuscated)	Emitted to all clients subscribed to a specific user's account whenever any account field is modified via a PATCH endpoint (status, privilege, username, email, or password).

forceLogout	string (reason message)	Emitted only to the targeted user's connected socket(s) when their account status is set to 'Inactive' by an Administrator. The client must immediately clear its auth state and redirect to the login page.
--------------------	-------------------------	--

10.3 Client-to-Server Events

Event Name	Payload	Description
subscribeAccount	username: string	Sent by a client to indicate it is viewing/editing a specific user's account. The server adds the socket to a room named account:<username> so it receives accountUpdated events for that account. Authorization is enforced: Members can only subscribe to their own account; Administrators can subscribe to any.
unsubscribeAccount	username: string	Sent by a client to stop receiving real-time updates for a specific account. The server removes the socket from the account:<username> room.

10.4 Room Strategy

- Each user account has a Socket.io room named account:<username>.
- When a client sends subscribeAccount("jdoe"), the server joins that socket to room account:jdoe.
- When a REST PATCH endpoint successfully updates an account, the controller emits accountUpdated to room account:<username> with the updated IUserAccount (password obfuscated).
- For forceLogout, the server emits directly to all sockets belonging to the inactivated user (identified by the JWT payload in their socket connection), not to the room — this ensures only the affected user's sessions receive it.

10.5 Event Flow by Endpoint

REST Endpoint	Socket.io Event Emitted	Target
PATCH /users/:username/status	accountUpdated to room account:<username>	All subscribers
N/A	forceLogout (if status → 'Inactive')	Inactivated user's sockets only
PATCH /users/:username/privilege	accountUpdated to room account:<username>	All subscribers
PATCH /users/:username/username	accountUpdated to room account:<oldUsername>, then room is renamed to account:<newUsername>	All subscribers
PATCH /users/:username/email	accountUpdated to room account:<username>	All subscribers
PATCH /users/:username/password	accountUpdated to room account:<username>	All subscribers
GET /users/:username	(none — read-only)	—

10.6 Force Logout Behavior (ties to Section 5 — R5)

When an Administrator sets a user's status to 'Inactive' via PATCH
/account/users/:username/status:

1. The REST response is returned to the Administrator with StatusUpdated.
2. The server emits accountUpdated to room account:<username> so any client viewing that account sees the status change.
3. The server emits forceLogout with reason "Your account has been deactivated by

- an administrator" to all connected sockets belonging to the inactivated user.
4. The server disconnects those sockets after a short delay (e.g., 500ms) to allow the client to process the event.
 5. The EmailServerBoundary service sends an inactivation notification email to the user (per the VOPC diagram).

On the client side, upon receiving forceLogout:

1. Clear JWT token from sessionStorage and localStorage.
2. Clear stored username from localStorage.
3. Redirect to the login page (auth.html).